

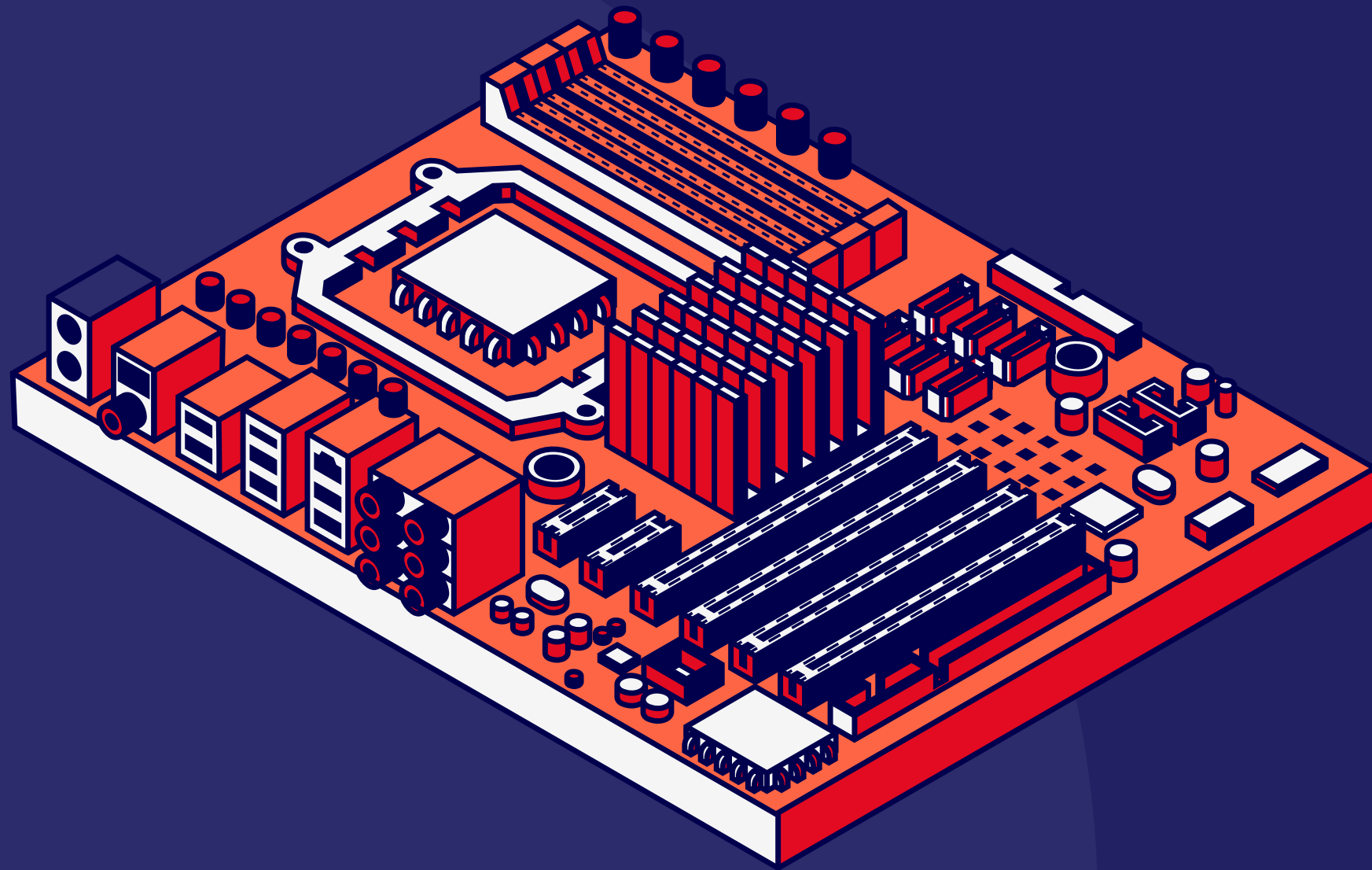
Parallel Algorithms for Butterfly Computations

Paper Presentation

By Awais, Sarmad, and Abdullah

Parallel Computing





Problem Statement

- Butterflies are the smallest non-trivial subgraph in bipartite graphs: a $(2,2)$ -biclique.
- Applications include fraud detection, document clustering, and dense-subgraph discovery in bipartite networks.
- Goal: Efficiently count butterflies at scale in large bipartite graphs.

PARBUTTERFLY Framework

- Modular framework supporting: global, per-vertex, and per-edge butterfly counting, plus peeling (tip/wing decomposition).
- Preprocessing: rank vertices (degree/coreness ordering) and sort neighbors.
- GET-WEDGES: parallel enumeration of all 2-paths (wedges).
- GET-FREQ: aggregate wedge frequencies to derive counts.





Parallelization Strategy

METIS Partitioning

- Split the graph into P parts, minimizing edge-cut and balancing work.

MPI Inter-Node

- Each rank loads its partition, then halo-exchanges ghost-neighbor lists via non-blocking MPI_Isend/MPI_Irecv.

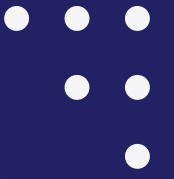
OpenMP Intra-Node

- `#pragma omp parallel for reduction(+ : count)` over local vertices to enumerate and count wedges.

Challenges & Metrics

- **Partition Quality:** edge-cut ratio vs. balance factor.
- **Communication Overhead:** cost of halo-exchange at scale.
- **Memory Footprint:** storing ghost-node adjacency.
- **Load Imbalance:** skewed degree distributions leading to hotspots.





Phase 2 Roadmap

- Apr 21–24: Write and validate `metis_partition.py`; measure edge-cut & imbalance.
- Apr 25–28: Implement MPI+OpenMP code in C++/mpi/; test correctness on toy graphs.
- Apr 29–May 1: Run strong/weak scaling experiments; collect per-rank timing.
- May 2–4: Generate performance plots, finalize `/docs/final_report.md`, and prepare demo.

